

« Un bon ETL est invisible. Il tourne sans bruit, produit des données fiables, et personne ne pense à lui parce que rien ne casse jamais. »

CHAPITRE I – FONDAMENTAUX DE L'ETL

— CE QU'EST UN ETL —

ETL signifie Extract, Transform, Load. C'est le processus qui déplace les données depuis leurs sources vers une destination exploitable, en les transformant au passage pour les rendre cohérentes, propres, et utilisables.

Extract : extraire les données depuis une ou plusieurs sources hétérogènes – bases de données, fichiers, APIs, sites web, systèmes legacy. Les données sont récupérées telles qu'elles sont, sans modification, dans leur état brut.

Transform : nettoyer, normaliser, enrichir, et restructurer les données pour les conformer au modèle cible. C'est là que vivent les 80% du travail réel. C'est là aussi que se prennent les décisions les plus importantes – et les plus risquées si elles ne sont pas documentées.

Load : charger les données transformées dans la destination finale – entrepôt de données, base analytique, fichier, dashboard. Le chargement peut être complet (full load) ou incrémental (seulement les données nouvelles ou modifiées).

Un ETL n'est pas un simple script de copie de données. C'est un système qui garantit la fiabilité, la cohérence, la traçabilité, et la répétabilité du mouvement de données dans le temps.

— ETL VS ELT : LA DISTINCTION MODERNE —

L'ETL classique transforme avant de charger. Les données passent par une couche intermédiaire de transformation avant d'arriver à destination. Cette approche est adaptée quand les ressources de calcul sont contraintes ou quand les transformations sont complexes et doivent être maîtrisées avant le chargement.

L'ELT (Extract, Load, Transform) charge d'abord les données brutes dans un entrepôt puissant, puis les transforme directement dans cet entrepôt avec SQL ou des outils spécialisés. Cette approche est possible et souvent préférable quand la destination dispose d'une puissance de calcul importante – comme les entrepôts cloud BigQuery, Snowflake, ou Redshift.

En pratique, pour un freelance data travaillant avec des PME et des volumes modérés, l'ETL Python classique reste la solution la plus adaptée, la plus maîtrisable, et la plus déployable.

— LES TYPES D'ETL —

Le Full Load recharge toutes les données à chaque exécution. Simple à implémenter, idéal pour les petits volumes et les données qui changent entièrement. Inconvénient : ne conserve

pas l'historique et peut être lent sur les grands volumes.

L'Incremental Load ne traite que les données nouvelles ou modifiées depuis la dernière exécution. Requiert un mécanisme de détection du changement – timestamp de modification, identifiant auto-incrémental, hash. Plus complexe mais essentiel sur les grands volumes.

Le Streaming ETL traite les données en temps réel ou quasi-réel au fur et à mesure de leur production. Nécessite des outils spécialisés (Kafka, Spark Streaming). Réservé aux besoins réellement temps réel – la plupart des PME n'en ont pas besoin.

Le Change Data Capture (CDC) détecte les modifications directement dans les journaux de transaction de la base source. Très efficace mais requiert des droits d'accès spécifiques sur la source.

CHAPITRE II – L'EXTRACTION

— PRINCIPES DE L'EXTRACTION —

L'extraction doit être non destructive. Les données source ne doivent jamais être modifiées pendant l'extraction. Demande systématiquement un accès en lecture seule. Si quelqu'un te donne un accès en écriture sur la production, refuse-le ou ne t'en sers pas.

Stocke toujours les données brutes telles qu'elles sont avant toute transformation. C'est la couche raw – ta sauvegarde contre toute erreur de transformation. Si tu te retrouves à refaire l'ETL six mois plus tard, tu as les données originales sans avoir besoin de relire la source.

Horodate chaque extraction. Sache toujours quand les données ont été extraites, depuis quelle version de la source, et combien de lignes ont été récupérées. Ces métadonnées sont indispensables pour diagnostiquer les problèmes.

Traite la source comme fragile. Les structures changent sans prévenir. Les APIs modifient leurs réponses. Les fichiers arrivent avec des colonnes supplémentaires ou manquantes. Valide toujours la structure de ce que tu reçois avant de le traiter.

— LES SOURCES ET LEURS DÉFIS —

Bases de données relationnelles. L'encodage, le fuseau horaire, et les types de données varient selon le SGBD. Une date MySQL n'est pas une date PostgreSQL n'est pas une date SQLite. Teste toujours les types sur un échantillon avant de traiter le volume entier. Utilise des requêtes paramétrées, jamais de concaténation de chaînes.

Fichiers CSV et Excel. L'encodage est le premier piège – UTF-8, Latin-1, Windows-1252 coexistent souvent dans le même lot de fichiers. Le séparateur peut être une virgule, un point-virgule, ou une tabulation selon la locale du système qui a exporté le fichier. Les fichiers Excel ont souvent des lignes parasites en haut, des colonnes fusionnées, et des cellules avec des formules plutôt que des valeurs.

APIs REST. La pagination est la chose la plus souvent ignorée et la plus coûteuse à ignorer. Des données tronquées qui semblent complètes sont pires que des données absentes. Les rate limits bloquent les scripts mal conçus. Les tokens expirent. Les versions d'API changent. Gère chacun de ces cas explicitement.

Web scraping. Le DOM change sans prévenir. Les sites détectent et bloquent les scrapers agressifs. Les données ne sont pas toujours dans le HTML – souvent dans des appels API JavaScript qui se font après le chargement de la page. Vérifie toujours si une API officielle existe avant de scraper.

Systèmes legacy. Les données peuvent être dans des formats anciens – DBF, fixedwidth, EBCDIC. Les encodages sont parfois propriétaires. Les structures ne sont souvent documentées nulle part. Prévois du temps pour l'archéologie.

— MÉCANISMES DE DÉTECTION DU CHANGEMENT

La colonne `updated_at` ou `modified_at` est le mécanisme le plus simple. Si la source l'a, utilise-la. Extrait les enregistrements dont la date de modification est postérieure à la dernière extraction réussie.

L'identifiant auto-incrémental fonctionne pour les insertions pures. Si les enregistrements ne sont jamais modifiés après création, extraire les IDs supérieurs au dernier ID traité suffit.

Le hash de ligne calcule une empreinte de chaque enregistrement et détecte les modifications en comparant les hashes entre deux extractions. Plus coûteux mais fonctionne même sans colonne de modification.

La table de watermark maintient en base le timestamp ou l'ID de la dernière extraction réussie. Mise à jour uniquement quand l'ETL se termine avec succès, jamais en cas d'erreur. C'est la méthode la plus robuste.

CHAPITRE III – LA TRANSFORMATION

— PRINCIPES DE LA TRANSFORMATION

Ne transforme jamais les données source directement. Travaille toujours sur une copie. La donnée brute est intouchable.

Documente chaque règle de transformation avec sa justification. Dans six mois, personne ne saura pourquoi cette valeur est remplacée par celle-là, ni si c'était correct. Le commentaire qui documente la décision est aussi important que la transformation elle-même.

Ne prends jamais une décision ambiguë seul. Si le comportement attendu pour une valeur manquante ou une valeur aberrante n'est pas évident, valide avec le métier avant d'agir. Une décision fonctionnelle prise unilatéralement est une bombe à retardement.

Logue le résultat de chaque transformation critique. Nombre de lignes entrantes, nombre de lignes sortantes, nombre de rejets, raisons des rejets. Ces chiffres sont ta preuve de bon fonctionnement et ton outil de diagnostic en cas de problème.

— LES TRANSFORMATIONS STANDARD

Nettoyage des textes. Supprimer les espaces en début et fin de chaîne. Normaliser la casse selon le besoin du domaine. Remplacer les valeurs nulles textuelles (NA, N/A, null, none, -) par des vraies valeurs manquantes. Supprimer les caractères non imprimables et les caractères spéciaux parasites.

Conversion des types. Les dates sont le cas le plus complexe – les formats varient entre les sources et les locales. Parser en spécifiant le format explicitement plutôt qu'en laissant la librairie deviner. Les nombres stockés en texte nécessitent souvent un nettoyage des séparateurs et des symboles avant conversion. Les booléens peuvent être représentés par oui/non, vrai/faux, 1/0, true/false – mapper explicitement vers un type cohérent.

Déduplication. Distinguer les doublons exacts (toutes les colonnes identiques) des doublons fonctionnels (même entité, données légèrement différentes). La règle de déduplication dépend du métier – garder le premier, le dernier, ou l'enregistrement le plus complet. Cette règle doit être validée par le métier, pas décidée unilatéralement.

Gestion des valeurs manquantes. Identifier si l'absence signifie "inconnu", "zéro", ou "non applicable". Ces trois cas ne se traitent pas de la même façon. Remplacer par zéro une valeur qui signifie "inconnu" est une erreur analytique grave. Valider avec le métier avant d'agir.

Normalisation des identifiants et codes. Un code client qui existe avec et sans zéros de tête, avec et sans espaces, en majuscules et minuscules – c'est un seul client que le système traite comme plusieurs. Cette normalisation est souvent l'étape la plus impactante sur la qualité finale.

Jointures et enrichissement. Vérifier les cardinalités avant toute jointure. Une jointure sur une clé non unique côté droite multiplie les lignes silencieusement. Toujours comparer le nombre de lignes avant et après. Loguer le taux de matching pour chaque jointure de référentiel.

— GESTION DES ANOMALIES —

Une anomalie est une donnée qui ne correspond pas au comportement attendu – valeur hors plage, combinaison impossible, donnée temporellement incohérente. La réponse correcte n'est jamais de la supprimer silencieusement.

Trois options selon la nature et le volume de l'anomalie. Corriger quand la règle de correction est évidente, documentée, et validée par le métier. Flaguer quand l'anomalie est suspecte mais que l'action à prendre n'est pas claire – elle reste dans le dataset avec un indicateur `is_anomaly`. Rejeter quand l'anomalie rend la donnée inutilisable – elle part dans une table de rejets avec la raison, disponible pour analyse et retraitement.

Un taux de rejet qui augmente d'une exécution à l'autre est un signal d'alerte. Il signifie soit que la qualité de la source se dégrade, soit que le format a changé, soit qu'un cas non prévu est apparu. Traite-le avant que le problème devienne structurel.

— RÈGLES MÉTIER ET CALCULS DÉRIVÉS —

Les règles métier sont les calculs, les classifications, et les catégorisations qui transforment les données brutes en indicateurs compréhensibles par l'organisation. La marge brute, le délai de paiement, le score de risque, la catégorie RFM – ce sont des règles métier.

Chaque règle métier doit être documentée avec sa formule exacte, ses hypothèses, ses cas particuliers, et l'approbation de quelqu'un du métier. Une règle de calcul non documentée et non validée est une source de conflits garantie quand deux personnes calculent le même indicateur différemment.

Teste toutes les règles sur un échantillon dont tu connais le résultat attendu avant de les appliquer au volume entier.

Si le chiffre d'affaires calculé ne correspond pas au chiffre de la comptabilité, la vérité est dans la comptabilité.

CHAPITRE IV – LE CHARGEMENT

— PRINCIPES DU CHARGEMENT —

Valide avant de charger. Après la transformation mais avant le chargement, effectue un ensemble de contrôles de qualité. Le nombre de lignes est-il cohérent avec l'attendu ? Les valeurs clés sont-elles présentes ? Les agrégats correspondent-ils à une valeur de référence connue ? Un chargement de données incorrectes est souvent plus difficile à corriger qu'une absence de chargement.

Utilise des transactions. Un chargement qui échoue à mi-chemin laisse des données partielles qui sont souvent pires que pas de données. La transaction garantit que soit tout est chargé, soit rien ne l'est.

Gère l'idempotence. Un ETL idempotent peut tourner deux fois de suite sans corrompre les données. Il détecte les doublons, écrase ce qui doit être écrasé, et n'insère pas deux fois ce qui a déjà été inséré. Cette propriété est essentielle pour la robustesse opérationnelle.

Conserve un audit trail. Enregistre dans une table de log chaque chargement avec son timestamp, le nombre de lignes insérées, mises à jour, et rejetées, et le statut final (succès ou échec avec message d'erreur).

— STRATÉGIES DE CHARGEMENT —

Truncate and Load est le plus simple. Vide la table cible et recharge tout depuis zéro. Adapté quand le volume est gérable et que l'historique n'est pas dans la table cible mais dans une couche séparée. Risque : fenêtre de disponibilité nulle pendant le chargement.

Append Only ajoute des lignes sans modifier les existantes. Adapté pour les données immuables comme les transactions ou les événements. Simple mais nécessite une gestion explicite des doublons.

Upsert (Insert or Update) insère les nouvelles lignes et met à jour les existantes basé sur une clé unique. La stratégie la plus polyvalente pour les données qui évoluent. Nécessite une clé unique fiable côté source.

Slowly Changing Dimensions (SCD) est la stratégie pour les données de référentiel qui changent dans le temps – un client qui change de segment, un produit qui change de prix. Plusieurs variantes existent selon le besoin de conserver ou non l'historique des changements.

— GESTION DES ERREURS AU CHARGEMENT —

Les erreurs de contrainte – violation de clé unique, valeur non nulle manquante, type incompatible – signalent un problème dans la transformation. Elles ne doivent jamais être ignorées silencieusement.

Stratégie de rejet : les lignes qui violent les contraintes partent dans une table d'erreurs avec la raison de l'échec. Le chargement des lignes valides continue. Une alerte est générée si le nombre de rejets dépasse un seuil.

Ne jamais catché une exception de chargement sans la loguer. Un échec silencieux est indétectable jusqu'au jour où

quelqu'un remarque que les données n'ont pas été mises à jour depuis une semaine.

CHAPITRE V – PIPELINE, MONITORING ET GOUVERNANCE

— RENDRE LE PIPELINE ROBUSTE —

Un ETL robuste est conçu pour les cas qui n'arrivent pas souvent mais qui arrivent toujours un jour. La source est indisponible. Le format a changé sans prévenir. Un volume anormalement grand arrive. La connexion réseau est perdue à mi-extraction. Chacun de ces cas doit avoir une réponse explicite dans le code.

Retry avec backoff exponentiel sur les erreurs réseau transitoires. Timeout explicite sur toutes les connexions. Validation de la structure de la source avant de traiter le contenu. Circuit breaker pour arrêter proprement quand trop d'erreurs s'accumulent.

Idempotence à chaque étape. Pouvoir relancer une étape qui a échoué sans dupliquer les données et sans corrompre l'état intermédiaire. Cette propriété est difficile à ajouter après coup – conçois-la dès le départ.

— MONITORING ET ALERTES —

Un ETL sans monitoring est une boîte noire. Tu ne sais pas qu'il a planté jusqu'à ce que quelqu'un remarque que les données ne sont plus à jour.

Les métriques à monitorer à chaque exécution. Durée totale et durée de chaque étape. Nombre de lignes extraites, transformées, chargées, rejetées. Taux de rejet comparé à la moyenne historique. Volume extrait comparé à l'attendu (un volume beaucoup plus faible que d'habitude signale un problème en amont).

Les alertes à configurer. Échec complet du pipeline – notification immédiate. Taux de rejet supérieur à 5% – notification pour investigation. Durée d'exécution deux fois supérieure à la normale – notification pour optimisation. Volume extrait inférieur à 50% du volume habituel – notification pour vérification de la source.

Les alertes doivent aller quelque part – email, Slack, SMS selon l'urgence. Une alerte qui n'est vue de personne ne sert à rien.

— DOCUMENTATION ET GOUVERNANCE —

Un ETL non documenté est un ETL qui appartient à son auteur et à personne d'autre. Quand l'auteur part ou oublie, l'ETL devient une boîte noire que personne n'ose modifier.

La documentation minimale d'un ETL comprend. Une description du flux de données – d'où viennent les données, où elles vont, quelles transformations elles subissent. Le catalogue des sources avec leur accès, leur format, et leur fréquence de mise à jour. Le catalogue des règles de transformation avec leur justification et leur validation métier. Le guide de déploiement – comment installer, configurer, et lancer. Le guide de dépannage – les erreurs fréquentes et comment les résoudre.

La gouvernance des données commence avec l'ETL. C'est ici que se définissent les règles de calcul des indicateurs, les règles de déduplication, les standards de nommage.

Ces règles, une fois définies et documentées dans l'ETL, deviennent la référence de toute l'organisation.

— TESTER UN ETL —

Les tests les plus importants pour un ETL. Test de bout en bout sur un échantillon réduit – la chaîne complète depuis la source jusqu'à la destination sur 100 lignes représentatives. Test des cas limites – source vide, source avec uniquement des erreurs, source avec le volume maximum prévu. Test de régression – après chaque modification, vérifier que le comportement sur les données de référence n'a pas changé.

La table de référence de test est un ensemble de lignes avec des valeurs connues et des résultats attendus vérifiés manuellement. Après chaque transformation, le résultat calculé doit correspondre au résultat attendu. Si ce n'est pas le cas, c'est une régression à corriger avant déploiement.

VÉRITÉS DU TERRAIN ETL

La moitié des bugs ETL viennent d'un changement silencieux dans la source. Monitorer toujours le format et le volume de ce qui entre dans ton ETL.

Un ETL qu'on peut relancer sans crainte est un ETL en qui on a confiance. Un ETL qu'on ne relance qu'avec appréhension est un ETL mal conçu.

Les règles de transformation non documentées sont les mines de terrain de la data. Elles explosent au mauvais moment, quand quelqu'un essaie de comprendre pourquoi deux calculs du même indicateur donnent des résultats différents.

La qualité d'un ETL ne se voit pas quand tout va bien. Elle se voit le jour où quelque chose d'inattendu arrive et que le pipeline gère la situation proprement au lieu de corrompre les données silencieusement.

Un ETL livré sans documentation n'est pas un livrable. C'est une bombe à retardement dont toi seul connais le code de désamorçage.
